



Detecting Dialog-Related Keyboard Navigation Failures in Web Applications

Paul T. Chiou, Ali S. Alotaibi and William G. J. Halfond

University of Southern California, USA

{paulchio, aalotaib, halfond}@usc.edu

Abstract—The ability to navigate the Web via the keyboard interface is critical to people with various types of disabilities. However, modern websites often violate web accessibility guidelines for keyboard navigability with respect to web dialogs. In this paper, we present a novel approach for automatically detecting web accessibility bugs that prevent or hinder keyboard users' ability to navigate dialogs in web pages. An extensive evaluation of our technique on real-world subjects showed that our technique is effective in detecting these dialog-related keyboard navigation failures.

I. INTRODUCTION

The Web has become an essential part of many people's daily lives, enabling them to complete everyday and essential tasks online and access important information resources. The ability to carry out these activities with ease is important for everyone and especially the approximately 15% of the world's population with disabilities [1], over half of whom rely on these online capabilities as a societal lifeline [2]. Unfortunately, sites on the web pose many accessibility challenges for users with disabilities. As of 2022, over 96% of the top million homepages on the Web were found to violate the most widely used accessibility standards [3]. When web applications are poorly designed, they create barriers that threaten the goal of universal accessibility.

One of the most common and prevalent accessibility issues studied among accessibility practitioners are those related to the use of dialogs in web applications [4], [5]. Dialogs serve an important role in modern web applications, including: prompting users for confirmation of actions, providing warnings for various actions, and enabling logins via external services. Although dialogs are typically implemented by JavaScript based user interface frameworks, these frameworks have struggled to incorporate accessibility into their implementations (e.g., [6], [7], [8], [9]), leaving developers to properly account for these issues. In particular, dialogs often have Keyboard Dialog Failures (KDFs), where users with motor disabilities, whose assistive technology is based on using keyboards or keyboard emulation to navigate a user interface, are unable to properly navigate into or out of the user interface defined by a dialog, navigate within the dialog, or dismiss the dialog. Beyond the fact that existing web accessibility standards require dialogs to be accessible (e.g., [10]), KDFs can prevent keyboard-based users from accessing important parts of the page and make it harder for them to effectively and predictably interact with

its User Interfaces (UIs) [11], [12], undermining the goal of universal accessibility.

For software developers, it is challenging to detect KDFs for many reasons. First, a developer must systematically interact with the web app to find every dialog that it can generate at runtime. This is labor intensive since the UIs of modern web apps often contain many user controls. Second, the process of checking web accessibility guidelines (e.g., [10]) is non-trivial. For example, to check **non-init-out** KDFs [II-A] requires the developer to track which element triggered the dialog and ensure that when the dialog is dismissed, that control automatically advances to the next web element in the UI's navigation order. Lastly, all of these checks have to be done using the keyboard to navigate the web application's UI. Although using a keyboard (or assistive technology that emulates a keyboard) is the most common mechanism for users with disabilities to navigate a web page, it is far from a common mechanism for many developers who may therefore be unfamiliar with keyboard mechanisms or how to test their behavior. Altogether, these challenges make it difficult for developers to thoroughly and accurately identify KDFs in their web applications.

Both commercial and research groups have developed tool support to help developers find accessibility issues in their web applications. However, to the best of our knowledge, only one, *Axe DevTools Pro* (Axe) [13] targets KDFs. However, as we explain in Section II and show in Section V, this tool only targets a subset of known KDFs, can only do so with limited accuracy, and is not fully automated. Other types of techniques, such as acceptance testing frameworks (e.g., [14], [15]), although not targeted at KDFs could be used by developers to manually generate test cases that are tailored for each KDF they anticipate in a web page. However, these require custom tests for each dialog of each web page in order to be useful. Lastly, well-known automated web accessibility scanners (e.g., [16], [17], [18], [19], [20]) cannot automatically identify dialogs in web pages, nor are they capable of detecting KDFs if given a page with dialogs. The limitations of these approaches mean that detecting KDFs remains largely a manual effort that requires the developers to exhaustively interact with a web page and explore all of the possible ways a dialog can be activated and interacted with, which can be error-prone, incomplete, and unscalable.

In this paper, we present our approach for automatically detecting KDFs in web applications. Our technique is the first

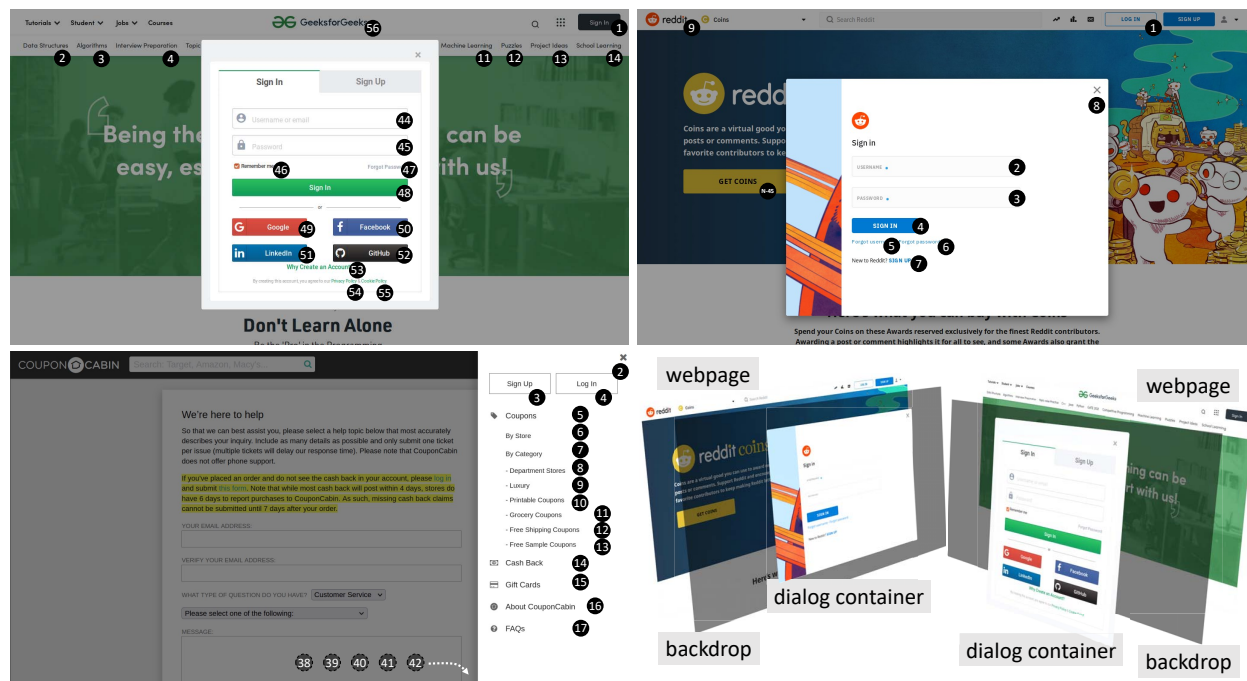


Fig. 1: Real-world examples of KDFs (*Geeksforgeeks* – top-left; *Reddit* – top-right; *CouponCabin* – bottom-left) and a breakdown of the design pattern of *modal* dialogs (bottom-right) that consists of a backdrop and a dialog container component.

fully automated detection technique for KDFs. To detect KDFs our approach analyzes the UI of a web page in order to model both keyboard navigation and dialog-related behavior. Next we defined detection algorithms that use this model to detect different types of KDFs. We implemented our approach as a prototype, LOTUS, to evaluate how well our technique could detect KDFs in real-world web applications. Our approach could detect all four types of KDFs with high recall, and all but one with very high precision. For the two subtypes that Axe could also detect using a semi-automated approach, our approach exhibited overall higher accuracy. The KDFs found by our approach were also highly impactful in the containing web application, in many cases requiring 30 times extra navigation effort or preventing access to key functionality of the web page. Lastly, many of the KDFs identified by our technique were either fixed or acknowledged by the developers of the web applications. Overall, these results show that our approach represents a very promising and practical technique for detecting an important class of web accessibility issues.

II. BACKGROUND

Keyboard usage is one of the foundational requirements for web accessibility and it is reflected in W3C Web Accessibility Initiative (WAI)'s Web Content Accessibility Guidelines (WCAG) 2.1 [21]. When interacting with a web page using a point-and-click (PNC) device, every web control element, (i.e., a hyperlink) is available to be accessed randomly. However, when interacting a web page using a keyboard-based device, such access is sequential and depends on the keyboard navigation sequence.

For example, during keyboard navigation, the user executes the `Tab` command to advance the keyboard focus to the next (or `Shift+Tab` to go back to the previous) control element. The actuation keys (`Space` or `Enter`) are then used to activate the element that is currently in focus (e.g., tick a checkbox or activate a dialog). When a user prompt or dialog is activated, the `Esc` key is then used to exit/dismiss the active dialog.

A *dialog* informs users about a task and can contain critical information or require decisions [22]. It is commonly used in modern single-page applications (SPAs) to display information on the same web page without unnecessary page reloads. Based on UI/UX design principle [23], a dialog can be classified as either a *modal* or a *modeless* dialog. A *modal* dialog is defined when the current UI is in a special state/mode that disables the main page content outside of the active *modal* dialog window until it is attended to. For web *modal* dialogs, the common UI pattern design visually obscures or dims the background page beneath as shown in the three examples in Figure 1. On the other hand, a *modeless* dialog's design does not obscure the background because it allows users to continue interacting with the background page content while the dialog is open.

Web dialogs can be triggered via keyboard navigation as well as via PNC navigation. However, due to the sequential (non-random) nature of keyboard-based navigation, the way a dialog's content is added/removed to/from the keyboard navigation flow (with respect to the entirety of the page) needs to be seamless and intuitive. Incorrect implementations

of dialogs can break the linearity of keyboard interaction and result in a navigation that is not sequentially consistent or intuitive with the meaning of the content. According to W3C, losing one's place in the content during navigation can negatively affect users who have limited short-term memory, perceive a narrow field of vision, use a screen magnifier, or require a physically challenging way to perform keyboard actions, such as using their eyes or tongue. [24].

In this paper, we focus on all the three general types of KDFs that are defined under WCAG Success Criterion (SC) 2.4.3 [10] and ARIA Authoring Practices Guide (APG) [25]. These are all of the KDFs defined by W3C. None of these KDFs have been directly addressed by state of the practice or state-of-the-art tools.

A. Focus Non-Initialization In/Out Dialog

Non-initialization type KDF in general, occurs when dialogs are introduced or discarded during keyboard navigation and the keyboard focus is not positioned with respect to the control that activated the dialog. According to WCAG, dialogs' position in keyboard navigation has to be adjacent to their trigger control in the sequential navigation order [26].

Non-init-in KDFs manifests when the focus is not set to an interactive element contained in the dialog when a dialog is activated. In Figure 1's *Geeksforgeeks* example, the *modal* dialog shown is triggered by the "Sign In" button (❶). However, instead of automatically directing the keyboard focus to the "Username or email" input (❷) upon triggering the dialog, the focus continues to follow the subsequent elements in the background page following the trigger element in the Document Object Model (DOM) (❶ → ❷ → ❸ → ...).

Non-init-out KDFs manifests when the focus is not returned to the element that invoked the dialog after the dialog is dismissed [27]. In the *Geeksforgeeks* example, when this dialog is dismissed, the keyboard focus should automatically return back to the element that triggered the dialog (❶). However, in this example, the immediate element to receive focus after dismissing the dialog is the "Geeksforgeeks" page icon (❸) at the beginning of the page's DOM.

When the focus is not directed to/from the dialog when it is inserted to the web page's navigation, the linearity of the flow is violated, which can confuse non-sighted keyboard users.

B. Focus Non-Containment Inside Dialog

Non-containment KDFs occurs during keyboard navigation when the keyboard focus is not limited to the elements within an activated *modal* dialog. Recall that *modal* dialogs are designed to overlay and interrupt the current web application process to prompt users a required response. Thus, it should prevent users from interacting with the content behind the *modal* dialog until it is dismissed. According to W3C, *modal* dialogs should not provide means for moving keyboard focus outside the dialog window without closing the dialog [28], [29]. In Figure 1's *Reddit* example, the *modal* dialog shown is triggered by the "Log In" button (❶). Upon activating the dialog, the keyboard focus is automatically set to the first

interactive element (❷) inside the dialog (in this example, a focus non-initialization KDF does not exist). However, the focus navigation is not confined within the dialog itself. When the user navigates forward via `Tab` through the contained elements, instead of looping from the last focusable element in the dialog (❸) to the first focusable element in the dialog (❷), the focus continues to follow the elements in the DOM, exits the dialog, and loops back to the beginning of the background page to the "Reddit" page icon (first focusable element ❹), all while the dialog is still open. Such non-containment KDFs may cause screen-reader keyboard users to become disoriented or confused when the content can be read from behind a *modal* dialog window.

C. Non-Dismissable Dialog

Non-dismissable KDFs occurs during keyboard navigation when a dialog cannot be dismissed after it is activated [30]. According to W3C, a dialog should be dismissable via either activating `Esc` or a designated button in the dialog [31]. There are two possible ways a non-dismissable KDF can manifest. The first way is when a keyboard user cannot dismiss the dialog by pressing `Esc` when navigating inside the dialog. The second way is when a designated UI control that is used to dismiss the dialog is not accessible to the keyboard. In Figure 1's *CouponCabin* example, the *modal* dialog's customized dismiss button `[x]` (❷) is implemented using a `<span>` element, which is by default, not accessible to the keyboard. While a PNC user can dismiss the dialog by mouse-clicking on it, a keyboard-based user would not be able to do so via a keyboard. In addition, in this example, no mechanism is implemented to dismiss the dialog (e.g., pressing `Esc`) on the elements within the dialog (❷ - ❸). This *modal* dialog therefore, remains open and can trap the keyboard focus [32], which prevents the user from returning to the main page's content.

III. MODELING WEB KEYBOARD NAVIGATION

In this section, we explain our approach for creating a model of the keyboard navigation and dialog properties of the Page Under Test (PUT). Our approach first analyzes the PUT to construct a model, which we call the Keyboard Dialog Flow Graph (KDFG), that represents the different ways a keyboard user can navigate a web page's UI. Our approach then analyzes the KDFG to identify dialogs that can appear in the PUT.

A. Modeling the Keyboard Dialog Flow Graph

The keyboard navigation and dialog-related properties of a PUT are represented by the Keyboard Dialog Flow Graph (KDFG). A KDFG is formally defined as a tuple $\langle G, v_0, \Phi \rangle$. In this tuple, G represents the set of keyboard navigation states the PUT can be in based on a user's keyboard inputs, v_0 represents the entry node of the PUT, which is the HTML element that receives the initial keyboard focus when the PUT is rendered in the browser, and Φ is the set of actions that could be carried out by a keyboard-based user (i.e., $\{\text{Tab}, \text{Shift} + \text{Tab}, \text{Enter}, \text{Space}, \text{Esc}\}$). Note that our

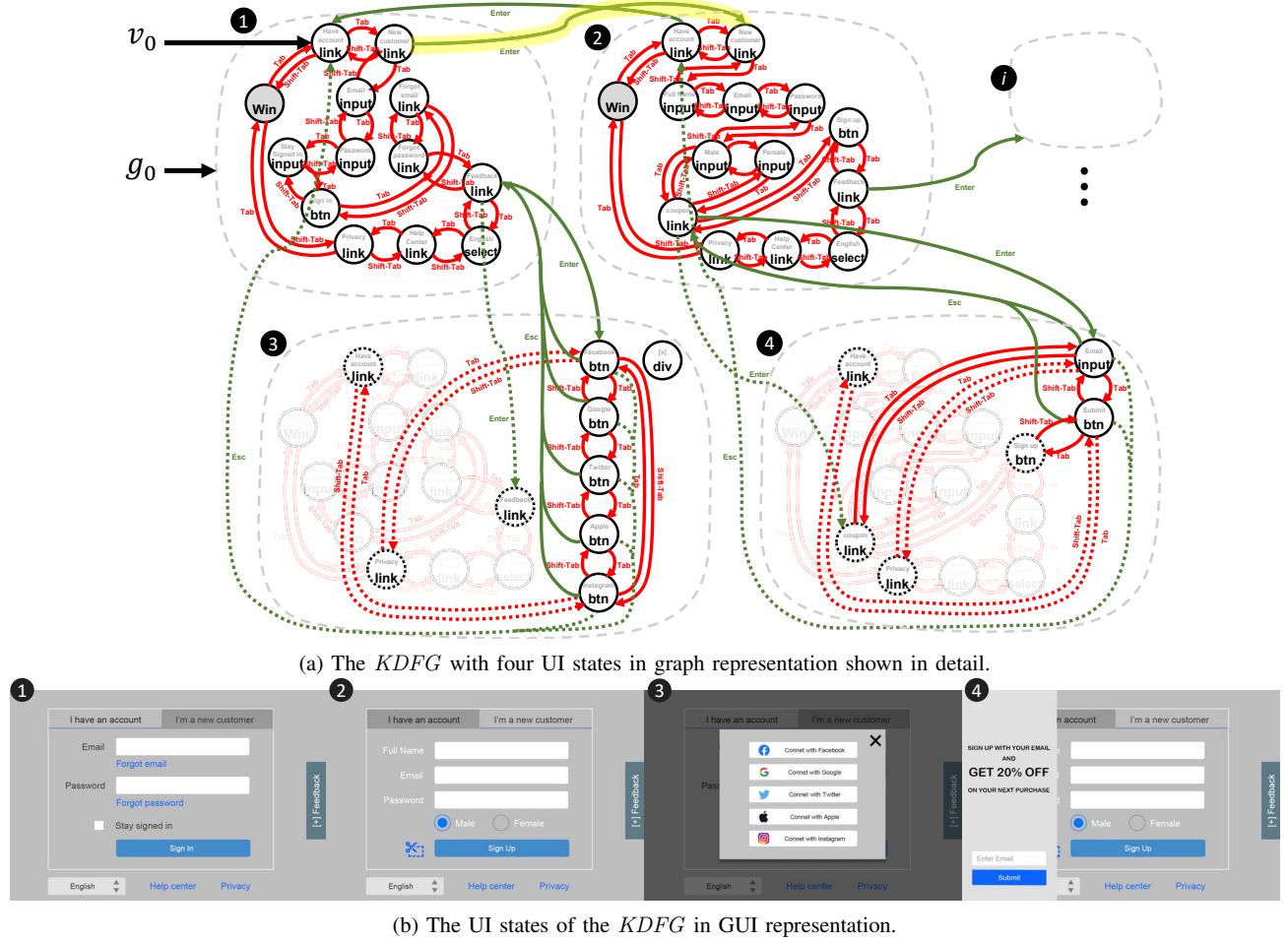


Fig. 2: A simplified version of the KDFG with their corresponding UI states where state ① (g_0) and state ② are states with no dialogs; state ③ contains a *modal* dialog; and state ④ contains a *modeless* navigation drawer type dialog.

approach does not define complex actions, such as inputting alphanumeric strings. These are not defined as part of the standardized keyboard accessibility testing process [33]). Each keyboard navigation state of the PUT $g \in G$ is itself a graph that represents both the HTML elements available to a user and the way a user can navigate among them using the keyboard. Formally, we define the state as a directed graph of the form $\langle V, E \rangle$, where V , the node set, represents the HTML elements visible in the state and E , the edge set, represents the corresponding keyboard navigation flow among the interactive elements in V .

A directed-edge $e \in E$ is defined as a tuple $\langle v_s, \phi, v_t \rangle$, indicating that when a source node v_s is in focus, the browser's keyboard focus shifts to a target node v_t by pressing key $\phi \in \Phi$. In the case where no new UI state results from executing the key press ϕ , the edge is an *intra-state edge*. In cases where ϕ causes a new set of HTML elements to be visible, the edge is an *inter-state edge* since the action also causes the user to transition to a new keyboard navigation state. Formally, we use the symbol $p(v)$ to represent the containing state of a node v ,

so $g = p(v)$ if $v \in g.V$. In the case of an *inter-state edge* $p(v_s) \neq p(v_t)$. The KDFG can be constructed using standard client-side crawlers that can support keyboard actions (e.g., Chiou et al. [34]).

An example of a KDFG is shown on the top in Figure 2 with the UI from which it is derived shown on the bottom. In this figure, we show each UI state (i.e., g) as a dotted bubble. We show *intra-state edges* in red and *inter-state edges* in green. Consider as an example, UI state ①, in which users can activate the “I’m a new customer” tab menu via `[Enter]` to expose a new set of controls (i.e., “Full Name” text input, etc.). The appearance of these new controls means a new *inter-state edge* has been identified that connects the link element *New Customer* in state ① to the link element *New Customer* in state ② and places the focus on the tab menu in the new state. We show this particular *inter-state edge* as yellow.

B. Identifying Web Dialogs

The first challenge for fully automated detection of KDFs is to identify dialogs in the PUT's UI. Identifying dialogs in an

automated way is challenging because there is no standardized way to implement dialogs in web pages. They can be implemented in a variety of ways, such as Bootstrap's *modal* [35], various lightbox plugins [36], the HTML5 `<dialog>` element [37], or static custom scripted content [38]. Since each framework uses its own technology and code structure, there are no implementation patterns that can be readily identified using static analysis techniques. Therefore, our approach defines mechanisms for identifying visual properties that are common to dialogs regardless of their underlying implementation mechanism.

Our approach analyzes the PUT's KDFG and produces a mapping $\mathcal{D}(g)$ that maps each UI state g to the set of dialogs contained in that state. To compute this mapping, our approach begins by performing a breadth-first traversal through all of the *inter-state edges* in the KDFG. Our approach iterates over only the *inter-state edges* because, by definition, the appearance of a dialog causes new elements to become visible, meaning that a new state has been defined. For every *inter-state edge* e , our approach identifies the new HTML elements that have become visible. The new elements are analyzed to see if they define a dialog (see Section III-B1 for details) and if so, then the dialog and page are analyzed to see if the dialog is *modal* or *modeless* (see Section III-B2 for details). By comparing the states of each *inter-state edge*, our approach can identify which add or remove dialogs and use that information to define $\mathcal{D}(g)$.

1) *Identifying Overlaying Dialog Container*: This part of the approach analyzes a set of elements that have been added to the page to determine if they represent a dialog. There are two parts to this analysis. In the first part, our approach identifies whether a candidate dialog container has appeared. A dialog container is an element that is able to contain the visual elements of a dialog. In the second part, our approach determines if the candidate dialog container overlays (i.e., appears in front of) the other elements in the page's UI.

Identify Candidate Dialog Container: To identify a candidate dialog container, our insight is that regardless of the underlying framework used to implement a dialog, the end result must cause an HTML element (e.g., `div` or `iframe`) to appear that itself contains the visual elements of a dialog. This element must also become (in)visible to make the dialog (dis)appear. In HTML/CSS, such elements will have certain properties that our approach can leverage to identify the dialog container. The first such property is that a dialog container must have a block-based property. Such a property renders a container-like structure that has properties like width, margin, border, height, and padding, which specifies the display behavior of a dialog. Therefore, our approach can identify (1) any element that changes visibility whose `display` property changes from `hidden` to `block`, or (2) any block-based element whose `hidden` property changes from `true` to `false`. Since there may be multiple such block-level elements within a dialog (depending on the web implementation), our approach takes the candidate dialog container with the largest Minimum Bounding Rectangle (MBR). This ensures the container is the one that encloses all of the content inside of the dialog.

Overlaying Rendering: After identifying a potential dialog container, the approach checks if it overlays the other elements in the page. This check takes advantage of the almost universal practice of rendering dialogs in front of other elements. Simply analyzing the container's `z-index` property is insufficient because the painting orders defined by the CSS Positioned Layout [39] do not necessarily render elements with larger `z-indexes` above lower ones. Therefore, we designed a process to model the way a browser's rendering engine draws the layers of visible elements on the screen, which we call the Stacking Model (SM). At a high level, the SM represents the standard stacking rules [40] used to create a hierarchy of available stacking contexts for the UI. Given two elements, the SM determines which element dominates (renders above) the other by comparing their stacking order with respect to their stacking contexts formed along the way up the DOM tree structure. Our approach first identifies those elements in the preceding UI state whose MBR collides with the container's MBR in the succeeding UI state. Our approach then uses the SM to determine if the newly visible elements in the container are stacked on top. If an element in the container is rendered on top of another element that is at its location, then our approach considers the container to be a dialog.

2) *Identifying Dialog Backdrop*: To determine whether an inserted dialog is a *modal* or a *modeless* type dialog, our approach checks for the existence of a backdrop component. Most implementations of *modal* dialogs follow the UI/UX design principle of rendering a *modal* dialog in front of a semi-transparent backdrop that overlays the browser's viewport. The intent of a backdrop is to completely fade out all elements of the page to provide a visual cue that a *modal* dialog requires an action in order for the page to proceed [41]. Our approach identifies a backdrop if it satisfies the size and transparency properties described below.

Size Property: Since a backdrop will obscure all elements of a page, the MBR of a backdrop element will cover almost the entire browser viewport. Therefore, our approach examines each potential backdrop element to see if its size is within a threshold of the browser's viewport size. Our approach utilizes a threshold to account for when (1) scroll bars may still be visible in the browser, and (2) when the backdrop element is set to full-width to fit within a `max-width` container of a navigation panel [42]. We looked at 100 web pages with *modal* dialogs and found that standard scrollbars typically take up 2% of the page's width/height and customized ones up to 4-5%. Therefore, we chose 90% as a safe upper bound of the maximum width/height, which did not introduce any inaccuracy into our analysis.

Transparency Property: Our approach also checks whether a possible backdrop element is a UI component that visually exhibits semi-transparency. The insight for this is that, by design convention, a *modal* dialog's backdrop is semi-transparent in order for the obscured background UI to be semi-visible. Based on the rules of web rendering, an element can be transparent if it has a final computed CSS (1) `opacity` property or (2) `background-color`

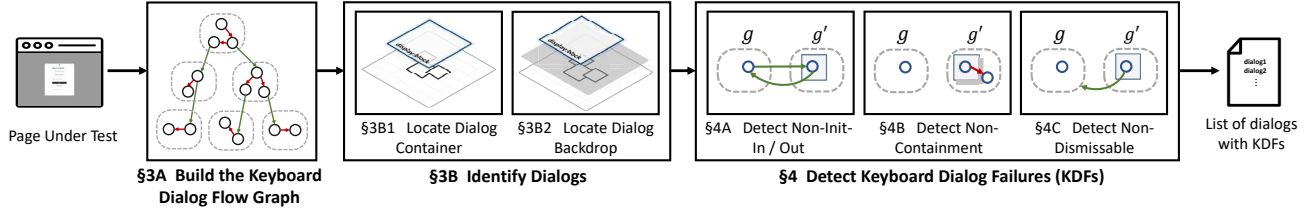


Fig. 3: An overview of the approach.

property with an RGBA alpha value of less than 1.0 (100%). In most cases, the transparency of an element can be detected by these two CSS attributes. However, developers may also use CSS transitions to animate the backdrop's transparent appearance over a time interval to exhibit a fading effect. Thus, our approach also checks for CSS properties, such as `will-change` and `transition-property`: that alter the element's opacity during the state transition.

IV. DETECTING KEYBOARD DIALOG FAILURE

To perform the detection of Keyboard Dialog Failures (KDFs), the approach analyzes (1) the keyboard navigation based on our KDFG model in Section III-A; and (2) the dialogs identified based on the mapping $\mathcal{D}$ from Section III-B.

A. Detecting Focus Non-Initialization KDFs

To detect non-initialization type KDFs, whose behaviors are described in Section II-A, the approach uses the formalized detection algorithm as shown in Algorithm 1. Recall that non-initialization type KDF can be either: “non-init-in” or “non-init-out” failures. They represent when the keyboard focus is not directed into or from a dialog when a dialog is inserted or removed from the PUT's keyboard navigation. We explain the detection of each in detail in the following paragraphs.

To identify “non-init-in” failures, the intuition is to see if the keyboard focus does not move inside of the dialog after it is activated. In order for an “non-init-in” failure to occur, three conditions must hold true. The first condition is that a dialog has to appear in the UI. We ensure this by only examining those edges that represent a UI state transition where a dialog is added. These edges are identified where the number of dialogs in its target state is more than those in its source state (line 4). The second condition is that the focus after the dialog appears is not on a node that is inside of the dialog. To check for this, we first identify the newly inserted dialog d by taking the difference between the set of dialogs in the target state with those in the source state (line 7). Once d is identified, we then find all the nodes that are inside (i.e., V_d), and check if the target node of the edge that triggered the dialog state is in V_d (line 10). Per W3C, if the focus does not automatically shift onto a dialog node, it is still considered accessible as long as the user can navigate sequentially onto a dialog node [43]. Therefore, the third condition is for the focus not to be in the dialog after advancing the focus in the sequential navigation via `Tab`. We check for this by examining the target node of the UI transition edge to see if it does not have an outgoing edge

with $\phi = \text{Tab}$ whose target node is inside V_d (line 12). If all three conditions apply, then the inserted dialog is considered to have a “non-init-in” type failure (line 13).

To identify “non-init-out” failures, the intuition is to check, after the dialog is dismissed, if the focus does not return to the original trigger that activated the dialog. In order for an “non-init-out” failure to occur, three conditions must hold true. The first condition is that a dialog has to be dismissed in the UI. This behavior can be identified by examining those edges that represent a UI state transition where a dialog is removed when the number of dialogs in its source state is more than those in its target state (line 5). The second condition is for the focus not to return to the trigger node that originally opened the dialog that is dismissed. To check for this, the algorithm first identifies the edge that originally inserted the dialog as e_ϕ (line 15). The goal of finding e_ϕ is to find the original trigger node v_ϕ that activated the dialog. v_ϕ is identified as the source node of e_ϕ (line 17). The algorithm checks if the target node of the edge that dismissed the dialog is not v_ϕ , which implies that the returned focus is not on the original trigger node (line 19). In the case where multiple cascading dialogs may be dismissed with one action, the approach selects the first inserted dialog as the one to check for. This is because this first dialog's trigger node is the point of return into the main page for the set of these multiple cascading dialogs. Following W3C's principle, the third condition is for the focus not to be put on v_ϕ after advancing the focus backward in the sequential navigation order via `Shift+Tab`. In other words, it checks if the element in focus after the dialog is dismissed (i.e., $e_{out}.v_t$) does not have an outgoing edge with $\phi = \text{Shift+Tab}$ whose target node is v_ϕ (line 21). A dismissed dialog is considered to have a “non-init-out” type failure if it satisfies all three conditions (line 22).

In Figure 2, non-initialization KDF edges are portrayed with dotted *inter-state (green) edges* (e.g., between state ❶ and state ❸) that either do not direct the focus from the dialog trigger $v_{feedback}$ in state ❶ into one of the six elements in the dialog in state ❸ or vice versa. The solid *inter-state (green) edges* between state ❶ and ❸ represent the ideal conforming behavior where the focus is properly initiated inside of the dialog and out back onto the dialog trigger.

B. Detecting Focus Non-Containment KDFs

To detect non-containment type KDFs, whose behaviors are described in Section II-B, the goal is to identify whether the keyboard focus is not confined to the elements within

Algorithm 1 Detecting Focus Non-Initialization KDFs

Input: $KDFG$: Keyboard navigation model
Output: $F_{init-in}, F_{init-out}$: Sets of dialogs with non-init-in and non-init-out KDFs

```

1:  $F_{init-in} = \emptyset$ 
2:  $F_{init-out} = \emptyset$ 
3:  $E_{inter-state} \leftarrow$  inter-state edges  $e$  in  $KDFG$  where  $p(e.v_s).V \neq p(e.v_t).V$ 
4:  $E_{init-in} = \{e \mid e \in E_{inter-state} \wedge |\mathcal{D}(p(e.v_s))| < |\mathcal{D}(p(e.v_t))|\}$ 
5:  $E_{init-out} = \{e \mid e \in E_{inter-state} \wedge |\mathcal{D}(p(e.v_t))| < |\mathcal{D}(p(e.v_s))|\}$ 
6: for all edge  $e_{in}$  of the form  $\langle v_s, \phi, v_t \rangle \in E_{init-in}$  do
7:    $d \leftarrow \mathcal{D}(p(e_{in}.v_t)) \setminus \mathcal{D}(p(e_{in}.v_s))$   $\triangleright$  /* the inserted dialog */
8:    $V_d \leftarrow d.nodes$ 
9:   /* check if focus is in the dialog */
10:  if  $e_{in}.v_t$  is not in  $V_d$  then
11:    /* check if advancing the focus puts focus in the dialog */
12:    if  $e_{in}.v_t$  has no out edge with  $\phi = \text{Tab}$  whose target node is in  $V_d$  then
13:      add  $d$  to  $F_{init-in}$ 
14: for all edge  $e_{out}$  of the form  $\langle v_s, \phi, v_t \rangle \in E_{init-out}$  do
15:    $e_\phi \leftarrow$  the  $e_{in}$  edge that originally inserted the dialog that is dismissed by  $e_{out}$ 
16:    $d \leftarrow \mathcal{D}(p(e_\phi.v_t)) \setminus \mathcal{D}(p(e_\phi.v_s))$   $\triangleright$  /* the dialog that is dismissed */
17:    $v_\phi \leftarrow e_\phi.v_s$ 
18:   /* check if focus is on the trigger */
19:   if  $e_{out}.v_t$  is not  $v_\phi$  then
20:     /* check if advancing the focus backward puts focus on the trigger */
21:     if  $e_{out}.v_t$  has no out edge with  $\phi = \text{Shift-Tab}$  whose target is  $v_\phi$  then
22:       add  $d$  to  $F_{init-out}$ 

```

an activated *modal* dialog during keyboard navigation. The formalized detection algorithm is shown in Algorithm 2. In order for a non-containment type KDF to occur, two conditions must hold true. The first condition is for a *modal* dialog to exist. To check for this, we first identify all of the *inter-state* edges that add a dialog to the UI as a result of their state transition (this is the same set of edges that is found in line 4 of Algorithm 1). For every such inserted dialog d , the approach checks for the existence of a backdrop to determine if it is a *modal* dialog (line 6). The second condition is for there to exist a navigation that would take the keyboard focus from a node within the dialog to a node that is outside of the dialog. To check for this, the approach first identifies the set of nodes V_d that are contained in the *modal* dialog (line 7). The approach then finds the set of complementary nodes $\bar{V}_d$ in the UI state by taking the difference between the set of nodes in the visible state and those that are in the dialog (i.e., V_d) (line 8). $\bar{V}_d$ represents the set of nodes outside the *modal* dialog. Next, the approach iterates through the edge set of this *modal* dialog's state to identify those *intra-state* edges whose source node is in V_d and whose target node is in $\bar{V}_d$ (lines 10-12). Any such edge exemplifies that the keyboard focus is not bounded within the dialog's control elements, thus a non-containment failure.

Algorithm 2 Detecting Focus Non-Containment KDFs

Input: $KDFG$: Keyboard navigation model
Output: $F_{contain}$: Set of dialogs that contain non-containment KDFs

```

1:  $F_{contain} = \emptyset$ 
2:  $E_{inter-state} \leftarrow$  inter-state edges  $e$  in  $KDFG$  where  $p(e.v_s).V \neq p(e.v_t).V$ 
3:  $E_{init-in} = \{e \mid e \in E_{inter-state} \wedge |\mathcal{D}(p(e.v_s))| < |\mathcal{D}(p(e.v_t))|\}$ 
4: for all edge  $e$  of the form  $\langle v_s, \phi, v_t \rangle \in E_{init-in}$  do
5:    $d \leftarrow \mathcal{D}(p(e.v_t)) \setminus \mathcal{D}(p(e.v_s))$   $\triangleright$  /* the inserted dialog */
6:   if  $d.backdrop$  is not empty then
7:      $V_d \leftarrow d.nodes$ 
8:      $\bar{V}_d \leftarrow p(e.v_t).V \setminus V_d$ 
9:     /* check the modal dialog state for crossing edges */
10:    for all edge  $e_{inState}$  of the form  $\langle v_s, \phi, v_t \rangle \in p(e.v_t).E$  do
11:      if  $p(e_{inState}.v_s) = p(e_{inState}.v_t)$  then
12:        if  $e_{inState}.v_s \in V_d \wedge e_{inState}.v_t \in \bar{V}_d$  then add  $d$  to  $F_{contain}$ 

```

In Figure 2, non-containment KDF edges are portrayed as the dotted *intra-state* (red) edges in state ③ that cross the focus from the first/last node inside the dialog into a node in the background page (i.e., $\bar{V}_d$). The solid *intra-state* (red) edges between $v_{facebook}$ and $v_{instagram}$ represent the ideal conforming behavior where the focus loops within the content of the *modal* dialog.

C. Detecting Non-Dismissable KDFs

To detect non-dismissable type KDFs, whose behaviors are described in Section II-C, the goal is to identify whether an activated dialog can be dismissed via keyboard navigation. The formalized detection algorithm is shown in Algorithm 3.

In order for a non-dismissable type KDF to occur, two conditions must hold true. The first condition is for there to be an existing dialog. This can be identified via an *inter-state* edge that inserts a dialog as previously mentioned in line 4 of Algorithm 1. The second condition is to check if the existing dialog can be dismissed. Recall that the dialogs in each UI state g are represented by the set $D = \mathcal{D}(g)$. The goal is to find if there exists a way to navigate from the UI state of an inserted dialog d to another UI state where d no longer exists in that state's D . To check for this, let the target node of the *inter-state* edge, which is the initial node to receive focus in the dialog state be denoted as v_{d0} (line 6). The approach then employs a graph reachability analysis to check if there exists a navigation path in the $KDFG$ that can lead the keyboard focus from v_{d0} to a UI state in which the inserted dialog is removed. In other words, our approach examines if there exists a node that is reachable from v_{d0} that also resides in a state in which the corresponding dialog is not present (lines 9-10). A dialog d is considered dismissed from a state g if it is no longer present in the state's set of dialogs D .

Algorithm 3 Detecting Non-Dismissable KDFs

Input: $KDFG$: Keyboard navigation model
Output: $F_{dismiss}$: Set of dialogs that contain non-dismissable KDFs

```

1:  $F_{dismiss} = \emptyset$ 
2:  $E_{inter-state} \leftarrow$  inter-state edges  $e$  in  $KDFG$  where  $p(e.v_s).V \neq p(e.v_t).V$ 
3:  $E_{init-in} = \{e \mid e \in E_{inter-state} \wedge |\mathcal{D}(p(e.v_s))| < |\mathcal{D}(p(e.v_t))|\}$ 
4: for all edge  $e$  of the form  $\langle v_s, \phi, v_t \rangle \in E_{init-in}$  do
5:    $d \leftarrow \mathcal{D}(p(e_{in}.v_t)) \setminus \mathcal{D}(p(e_{in}.v_s))$   $\triangleright$  /* the dialog under test */
6:    $v_{d0} \leftarrow e.v_t$ 
7:    $V_{reachable} \leftarrow$  nodes reachable from  $v_{d0}$ 
8:    $isDismissFailure = \text{true}$ 
9:   for all node  $v_{reachable} \in V_{reachable}$  do
10:    if  $v_{reachable}$  is in a state  $g.V$  and  $d \notin \mathcal{D}(g)$  then
11:       $isDismissFailure = \text{false}$ 
12:   if  $isDismissFailure$  then add  $d$  to  $F_{dismiss}$ 

```

In Figure 2, the dismissal node (i.e., $v_{[x]}$) in state ③ does not have navigation that can facilitate returning the focus from state ③ back to ①. This is because $v_{[x]}$ is both (1) unable to receive keyboard focus (i.e., not reachable via navigation); and (2) lacking the appropriate keyboard event handling that activates the dialog's dismissal (i.e., no outgoing edge with actions **Enter** or **Esc** coming out of $v_{[x]}$ that transitions to a state where the dialog is dismissed). State ③ can contain a non-dismissable type KDF if its set of six dialog nodes do not include any of the either dotted or solid *inter-state* (green)

edges whose $\phi = \text{Esc}$. The lack of these edges would indicate that the user would also not be able to dismiss the dialog via `Esc` to transition the UI out of the dialog state.

V. EVALUATION

To assess the effectiveness of our approach, we conducted an empirical evaluation that focused on the following four research questions (RQs):

RQ1 What is the accuracy of our approach in detecting and classifying the type of dialogs in web applications?

RQ2 What is the accuracy of our approach in detecting KDFs?

RQ3 How fast is our approach in detecting KDFs?

RQ4 What is the impact of the KDFs that our approach detected?

Details of the experimental data are available to the community via the project website [44].

A. Implementation

We implemented our approach as a Java-based prototype tool called `diaLOg accessibility failUres inSpector (LOTUS)`. Our approach employs Selenium (version 3.141.5) to build the KDFG model using Firefox 92.0. The WebDriver API [45] is used to simulate keyboard-based actions to interact with the running web page. To extract the web dialogs' properties from the UI, our approach executes JavaScript through WebDriver and employed the Intersection Observer API [46] to effectively capture the visible DOM at any point during runtime. The approach used XPERT's DOM model [47] to help build the UI representation used to construct our Stacking Model in order to determine the browser's painting operations. The evaluation was performed on an AMD Ryzen Threadripper 2990WX with 64GB memory running 64-bit Ubuntu Linux 18.04.4 LTS.

B. Subject Web Pages

We conducted our evaluation on a set of real-world subject web pages selected from both the Moz 500 top websites [48] and websites that are similar to the Moz 500 (e.g., sites returned from *similarsites.com*). The selection of these two sources follows a similar methodology to that of Bajammal et al. [49] to obtain a diverse set of sites that use different web technologies. We manually interacted with the web pages of random websites selected from these two sources to create the ground truth of dialogs and KDFs. We interacted with these pages until we found 30 pages that contained a KDF and 30 that did not. Out of the 60 subjects, 44 contained at least one dialog and the remaining 16 did not. We included subjects that do not contain dialogs or KDFs to help measure the false-positive detection rate for RQ1 and RQ2. Overall, our 60 subject web pages contain a total of 88 dialogs and represent 44 non-init-in/non-init-out, 43 non-containment, and 12 non-dismissable KDFs, as well as 35 dialogs with no KDFs. Note that some dialogs had more than one KDF, which is why the sum of the KDFs is greater than the total number of dialogs.

C. Ground Truths for Accuracy Evaluation

For building the dialog ground truth, the authors interacted with each element to determine if it could trigger a UI change, and then determined, based on the W3 WAI definitions/examples [50], if the UI change was a dialog and its type.

For building the KDF ground truth, the authors interacted with all of the dialog triggers to activate the dialogs. **For non-init-in KDFs**, we observed whether the keyboard focus (1) was immediately set to an element that was inside the dialog or (2) was going to be set to an element that is inside the dialog after navigating forward. **For non-init-out KDFs**, we checked if an open dialog, after being dismissed, returned the keyboard focus (1) immediately back to the trigger element or (2) back to the trigger element after navigating backward. **For non-dismissable KDFs**, we observed if an open dialog could be dismissed via either a dismiss button or the `Esc` key. If the dialog contained a non-init-in KDF, where the focus was not automatically set into the dialog, then we manually set the focus to a random element that was inside the dialog to proceed with testing the dismissal. **For non-containment KDFs**, we set the focus to a random element inside *modal* dialogs and continued to navigate forward or backward to observe whether the focus looped back into the dialog elements. The above process follows WCAG Techniques as outlined in SC 2.4.3 [51], which specifies the exact behaviors of the failures that a keyboard-based user would experience.

Both sets of ground truths were defined independently by the first two authors and there was no disagreement.

D. RQ1: Accuracy of Detecting and Classifying Web Dialogs

The goal of RQ1 is to assess the effectiveness of our approach in detecting and classifying dialogs. For dialog detection, we ran LOTUS on all of our subjects and computed its precision and recall with respect to the ground truth of dialogs identified in Section V-C. For dialog classification, we computed the precision and recall of LOTUS in determining if the identified dialogs were *modal* or *modeless*.

The results for RQ1 showed that our approach's heuristics could accurately detect and classify dialogs. Overall, LOTUS correctly identified dialogs across all of the subject web pages with 90% precision and 89% recall. Among those identified dialogs, LOTUS was able to correctly classify *modal* dialogs with 97% precision and 98% recall and *modeless* dialogs with 92% precision and 86% recall. In general, our approach was able to identify dialogs in the subject web pages with high accuracy.

To better understand situations where our approach was not accurate, we manually analyzed all of the incorrect results. We found that the dominant cause (5 cases) of false-positive dialog detection was the use of special sliding carousel widgets in the PUT. When these carousels rotated new content into view, the heuristics mistakenly identified the sliding carousel container as a dialog container. The main reason for false-negative dialog detections was actually due to the implementation of our evaluation's subject caching infrastructure. In two cases, it was not possible for our caching infrastructure to record certain

TABLE I: Results for KDF detection accuracy (RQ2) and run time (RQ3).

Tool	Non-Init-In		Non-Init-Out		Non-Containment		Non-Dismissable		Run-time	
	P	R	P	R	P	R	P	R	Model Building	Detection
LOTUS (<i>auto</i>)	71%	89%	71%	77%	70%	90%	40%	83%	26:41	00:02
LOTUS (<i>perfect</i>)	85%	94%	71%	87%	90%	93%	63%	83%	26:41	00:02
Axe (<i>trigger only</i>)	17%	10%	0%	0%	59%	36%	0%	0%	n/a	01:30
Axe (<i>full manual</i>)	45%	70%	0%	0%	59%	64%	0%	0%	n/a	03:00

properties in the DOM, therefore our approach could not properly load HTML nodes inside of the dialog. The remaining false-negative was a dialog that popped up to notify the users that they are leaving the current web page. The approach was not able to identify this particular notification dialog because the page automatically navigated away five seconds after it was displayed (our approach needed about 45 seconds to extract a state's UI nodes). In terms of the ability to distinguish the type of a dialog, we found one case where a navigation drawer *modeless* dialog was incorrectly classified as a *modal* dialog. This was because the *modeless* dialog had a frame with cosmetic shadow effects that exhibited transparency, which our approach misidentified as the existence of a backdrop component.

E. RQ2,RQ3: Accuracy and Runtime of Detecting KDFs

The goal of RQ2 and RQ3 is to evaluate the accuracy and runtime of our approach in detecting KDFs. To do this, we ran LOTUS against all of our subject web apps and computed its precision and recall for each of the four types of KDFs, and also measured the time required to perform the detection. To provide context for our results, we also compared against a well-known and popular accessibility auditing tool, *Axe DevTools Pro* (Axe) [13], which to the best of our knowledge is the only state-of-the-art technique capable of detecting some types of KDFs. To ensure we did not overlook any other tools for comparison purposes, we examined tools listed on W3C's *Web Accessibility Evaluation Tools List* [52] as well as prior web accessibility literature [34], [49] for additional tools to compare against. Based on this investigation, we found three tools, *WAVE* [20], *QualWeb* [53], and *SortSite* [54], which based on their descriptions indicated they might be able to detect some KDF related issues. We ran these three tools on our subject web pages and found that none of them were able to detect KDFs. Instead, these tools could only display ARIA-based warnings [55], [56] to indicate that the page contained dialog accessibility support for screen-readers.

The comparison against *Axe* (version axe-core 4.4.3) added some additional steps to our evaluation protocol. These were necessary to account for the fact that *Axe* is not fully automated and requires interventions by the developers to perform KDF detection. Specifically, *Axe* requires developers to provide it with the trigger node in a page's UI that causes the dialog to appear. *Axe* then triggers the dialog and interacts with it to determine if it has a KDF. In some cases when *Axe* is not able to find the dialog after activating the trigger nodes, it prompts the developer to highlight the specific parts of the

UI that represent the dialog. To address this, we added three additional sets of results to our evaluation. In the first set, which we denote as *Axe full manual*, we interacted with *Axe* to completely and accurately provide trigger nodes and then manually and accurately identified the dialog if the trigger node based identification failed. In the second set, which we denote as *Axe trigger only*, we provided only the trigger node and then did not provide further dialog identification assistance. For LOTUS we also added results that showed its detection accuracy if it could assume that its dialog detection heuristics were accurate (i.e., 100% precision and recall) so its detection accuracy could be more directly compared against *Axe* (i.e., *Axe full manual*). These results are denoted as *LOTUS perfect*. Note that since *Axe* requires developers to provide the dialog to be tested, it did not have meaning to evaluate *Axe* on the 16 subjects with no dialogs. Therefore, the precision and recall are only computed on the 44 subjects that contained dialogs. Also note that *Axe* does not detect non-init-out nor non-dismissable KDFs, therefore these results are counted as 0%. All results for RQ2 are shown in Table I. The columns "Non-Init-In", "Non-Init-Out", "Non-Containment", and "Non-Dismissable" show the average precision (P) and recall (R) across the set of all dialogs for each type of KDF for each of the four results.

The results for RQ2 show that our approach had high detection accuracy for all types of KDFs and its accuracy was significantly higher than *Axe*. Our approach's accuracy had an average F-score of 72% across the four types of KDFs for fully automated detection and this rose to 83% if the approach could assume perfect detection of the underlying dialogs. In contrast, for the scenario with fully accurate and complete manual intervention, *Axe* had an average F-score of 55% and 61% for the two types of KDFs it could detect. For those specific KDF types, our approach's F-score was 79% and 81% for the fully automated scenario, and 92% and 89% if the approach could assume perfect dialog detection.

We further analyzed the KDFs for which our approach had inaccurate results. In particular, we focused on understanding the reason that the precision for detecting non-dismissable dialogs was much lower than the precision for other KDFs. The underlying reason for this was due to the way edges in the KDFG that exited a dialog were tracked. Since each dialog was represented a single time in the KDFG, there were multiple exit edges, each denoting the exit from a different possible entry edge. When Algorithm 3 tracked reachable paths from an entry point to the dialog sub-graph, these extra edges resulted in false-positive detections. To a lesser extent, this also

affected the precision of non-init-out KDF detection. We note that this problem is similar to that faced by context-sensitive program analysis techniques and plan future research work to improve the modeling for this situation. For non-init-in and non-containment type KDFs, the most common reason for inaccuracies was due to the issues discussed in RQ1, inaccurate detection of nodes inside of the dialog. Lastly, we found 13 non-containment cases where the approach misidentified the node(s) that should be in the dialog versus those that were not in the dialog. As a result, an init-in edge or an edge that crossed the dialog's boundary was either missing or falsely constructed. Although we could not analyze the algorithms of *Axe* to the same depth as LOTUS, our observation was that limitations in how *Axe* interacted with the triggered dialogs was a significant contributor to its inaccuracies.

During our manual analysis, we also identified several common root causes for the KDFs in the subject web pages. Broadly, all of these could be attributed to missing or incorrect JavaScript for managing keyboard focus with respect to the dialog. For non-initialization types, when a dialog was activated in the web page, the responsible JavaScript did not insert the dialog at the DOM location of the dialog trigger. Instead, for example, it would append the dialog to the beginning or end of the DOM. For non-containment type KDFs, we found all of these failures' implementations lacked scripting altogether or did not contain any keyboard trapping mechanism to redirect the focus back into the *modal* dialog's controls. For non-dismissable type KDFs, the primary root cause was the use of custom components to close a dialog that did not have keyboard event handlers attached to them or could not be focused on using keyboard navigation.

The results for RQ3, runtime of the analysis per subject, are also shown in Table I. For LOTUS we broke the runtime down into two parts "*Model Building*", which is roughly the time spent doing the construction of the KDFG and identifying the dialogs it contains (i.e., Section III-A), and "*Detection*", which is the time to analyze the models and find the KDFs. For *Axe*, the dialog detection is done by the developers so we marked that part's time as "n/a." Overall, our detection is very fast and compares well with *Axe*. However, our model building time takes almost 27 minutes. Comparing this directly to *Axe* could be misleading, since the comparable time in the *Axe* detection process depends on how long a developer takes to identify each dialog in a subject and provide all of the corresponding trigger nodes. Unless the developer is completely familiar with the subject app, this task requires them to interact with every element in the PUT to locate dialog triggers as well as the dialog frames. For this reason, we conclude that the results of RQ3 show a reasonable runtime for our approach and would not be a barrier to its adoption.

F. RQ4: Impact of the Detected KDFs

The goal of RQ4 is to better understand how the observed KDFs impact keyboard users' ability to interact with web pages. To measure the impact of the KDFs, we employ a common approach in the accessibility research community

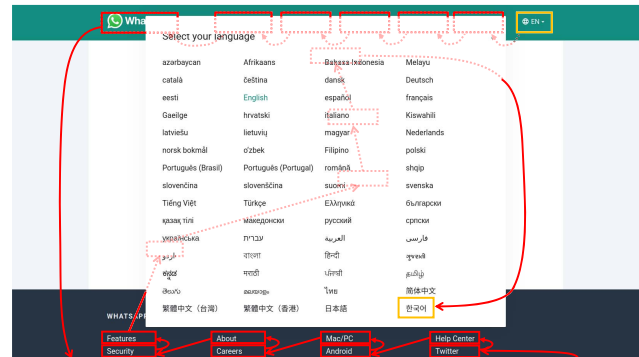


Fig. 4: A non-init-in KDF in *WhatsApp* that demonstrates a user has to navigate backward through the entirety of the page's content (adding more than 36 steps to the navigation) before the relevant language options in the *modeless* dialog are presented to them.

based on measuring the amount of effort necessary to work around, if possible, the accessibility issue [57]. For non-init-in, non-init-out, and non-containment type KDFs the workaround can be quantified and corresponds to how many times a keyboard-based user would need to press `Tab` or `Shift+Tab` to return to the right spot in the page's navigation. For non-dismissable, the impact for all of them was high since no workaround was possible.

To measure the impact of non-init-in and non-init-out KDFs, we measured the number of additional steps that it took for the keyboard focus to reach the dialog or return to the correct trigger element. Across our subjects, users would need an average of 27 extra keyboard steps (with a min, median, and max of 1, 14, and 82) to find a dialog after triggering it and 32 steps (with a min, median, and max of 1, 25, and 156) before they returned to the part of the page they were in before interacting with a dialog. For example, in the subject *TikTok*, after activating the "Log in" *modal* dialog, users had to sequentially `Tab` over 33 times through the page's interactive content in order to reach the dialog to log in.

To measure the impact of non-containment KDFs, we measured the number of steps for the focus to loop back into the dialog in either direction. Based on our results, it took an average of 54 extra keyboard steps (with a min, median, and max of 2, 55, and 161) to return to the dialog. In one egregious example, the subject *Vimeo* required over 161 `Tab` actions after the keyboard focus unexpectedly exited a non-contained "Create an account" dialog.

It was not possible for keyboard-based users to circumvent non-dismissable KDFs, which could limit users' ability to access core functionalities. For example, in the subject *OfferUp*, an auction e-commerce page, after a user downloaded the *OfferUp* app from a non-dismissable *modal* dialog, they were not able to return to the main page to list an item for sale.

G. Threats to Validity

A potential threat to external validity is that our experiment was run on Firefox version 92.0 and there could be

potential discrepancies in the way dialogs manifest between different web browsers and different versions. The browser-specific dialog manifestation and KDFs could be detected by systematically running LOTUS using different Selenium WebDriver drivers for different browsers and different versions.

A potential threat to construct validity is our operationalization of the concept of a KDF. In other words, does our working definition of a KDF match the intended idea. We believe that this is correct for several reasons. First, our definition of KDFs is based on the success criterion defined by WCAG 2.4.3 and APG, which are the international standards for web accessibility and one of the standards used to determine when a website is in violation of the ADA law [58]. The success criterion is defined to allow practitioners to objectively determine if code fails to satisfy accessibility requirements without the need to consult specialized accessibility auditors or conduct user studies. In addition to using the success criterion, we also reached out, via the websites' contact forms or provided email addresses, to the developers of our subject web pages and reported the KDFs we found. Overall, eleven developers acknowledged receiving our report, with five of them personally commenting on the importance of the findings, and another ten subjects were at least partially fixed when we checked their sites prior to the submission of this paper. Together, this represented positive validations for about two-thirds of the subjects that had KDFs. Altogether, we believe that our reliance on the WCAG success criterion, APG, the large measured impact of the KDFs our approach detected, and the positive validation we received, from either developers acknowledging the reported KDFs or fixing the KDF, shows that our operationalization of the KDFs construct is valid and useful.

VI. RELATED WORK

Web dialog accessibility has been a long-standing problem in the context of web accessibility. However, to date, no research has directly focused on its accessibility from the perspective of keyboard navigation. In the HCI field of study, dialog (or navigation) is often referred to as the interaction between users and the UI. Research on Self-adaptive UIs (SAUIs) has attempted to enhance usability by automatically adapting to the context-of-use at runtime [59] to reduce cognitive overload. Yigitbas et al. proposed a conceptual context-aware dialog model to automatically adapt web UIs to accommodate context variations, such as surroundings and user environments [60]. Studies have also presented browsing strategies via speech-driven interfaces to generate voice dialogs between humans and computers using NLP to assist users with visual impairments [61], [62]. While these techniques try to mitigate the ineffectiveness of existing speech-driven browsing by using content-based analysis segmentation of web page content, they do not address the accessibility issues pertaining to KDFs.

The closest work has shown that dialog interface can improve usability in keyboard-based interactions [63]; but this study only focuses on desktop applications and did not do detection. KAFE [34] proposed a technique to detect keyboard

accessibility issues in web applications. However, it tackles a different set of problems that prevents keyboard-based users from accessing important functionalities in a web page. While KAFE has shown that a web UI's keyboard accessibility can be quantified and abstracted through navigability, it does not account for the notion of dialogs and therefore cannot be used to detect KDFs. General strategies that address web accessibility include the detection of inaccessible layout properties [64], [65] and violations to accessible semantic structures [49]. These techniques specifically focus on accessibility relating to layout properties and are not capable of interacting with the PUT to expose KDFs. Current web accessibility scanners [16], [17], [18], [19], [20] are also inadequate in detecting these issues because they do not detect dialogs and they ignore the impact of the interactive behaviors that are related to dynamic content associated to dialogs.

Researchers have also examined different type of mobile apps accessibility issues [66], [67], [68], [69], and proposed techniques to repair these issues [70], [71], [72]. Alotaibi et al. proposed a technique to detect navigation issues related TalkBack [73]. Work by Salehnamadi et al. discussed cases where dialogs (such as navigation drawers) overlap with the background UI content, leading to inefficient navigation when using assistive services with Android apps [11]. This study executes UI test cases using assistive services to report an accessibility warning when a step in the use-case execution takes more than a specific number of interactions. These techniques are limited to the mobile app domain and do not focus on dialog accessibility nor can they detect KDFs.

There have also been studies that focus on detecting the components of mobile UIs [74] and define heuristics that can identify equivalent mobile UIs to better match Android activities with predefined templates [12]. Such techniques can help identify navigation drawers and floating dialogs specific to mobile apps, but cannot be applied to web apps.

VII. CONCLUSION

Web dialogs are intricate UI components in modern web applications that often lack considerations for accessibility. An ideal keyboard navigation of a web page should be in a meaningful sequence that is intuitive. However, many web design implementations result in an unpredictable keyboard navigation sequence that prevents or hinders keyboard users' ability to effectively interact with dialogs. In this paper, we proposed a novel approach to automatically detect dialogs as well as identifying Keyboard Dialog Failures (KDFs). Our evaluation shows our approach is able to accurately detect both dialogs and KDFs in real-world web pages. The results are overall positive and indicate that our approach can assist developers in debugging dialog-related keyboard accessibility issues in web applications.

ACKNOWLEDGMENT

This work was supported by the National Science Foundation under grant 2009045.

REFERENCES

- [1] "World Health Organization: World Report on Disability," https://www.who.int/disabilities/world_report/2011/report/en/, accessed: 2020-07-02.
- [2] "Almost half of people with disabilities don't use the Internet: but why? | Oxford Internet Surveys - OxiS." [Online]. Available: <https://oxis.oii.ox.ac.uk/blog/almost-half-people-disabilities-dont-use-internet-why/>
- [3] "The WebAIM Million: An annual accessibility analysis of the top 1,000,000 home pages," <https://webaim.org/projects/million/>, updated: 2021-11-10.
- [4] "DEV Community - How I approach keyboard accessibility for modals in React," <https://dev.to/colettwilson/how-i-approach-keyboard-accessibility-for-modals-in-react-152p>, accessed: 2021-08-24.
- [5] "Harvard Digital Accessibility - Technique: Accessible modal dialogs," <https://accessibility.huit.harvard.edu/technique-accessible-modal-dialogs>, accessed: 2021-08-24.
- [6] "GitHub Issues - angular/components," <https://github.com/angular/components/issues?q=dialog+accessibility>, accessed: 2021-08-24.
- [7] "GitHub Issues - reactjs/react-modal," <https://github.com/reactjs/react-modal/issues?q=dialog+accessibility>, accessed: 2021-08-24.
- [8] "GitHub Issues - react-bootstrap/react-bootstrap," <https://github.com/react-bootstrap/react-bootstrap/issues?q=dialog+accessibility>, accessed: 2021-08-24.
- [9] "GitHub Issues - vuetifyjs/vuetify," <https://github.com/vuetifyjs/vuetify/issues?q=dialog+accessibility>, accessed: 2021-08-24.
- [10] "WCAG - Understanding Success Criterion 2.4.3: Focus Order," <https://www.w3.org/WAI/WCAG21/Understanding/focus-order.html>, updated: 2021-08-25.
- [11] N. Salehnamadi, A. Alshayban, J.-W. Lin, I. Ahmed, S. Branham, and S. Malek, *Latte: Use-Case and Assistive-Service Driven Automated Accessibility Testing Framework for Android*. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: <https://doi.org/10.1145/3411764.3445455>
- [12] X. Zhang, A. S. Ross, and J. Fogarty, "Robust annotation of mobile application interfaces in methods for accessibility repair and enhancement," in *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology*, ser. UIST '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 609–621. [Online]. Available: <https://doi.org/10.1145/3242587.3242616>
- [13] "Deque: axe DevTools - The Accessibility Testing Toolkit For Developers," <https://www.deque.com/axe-devtools-accessibility-testing/>, updated: 2021-11-23.
- [14] W. M. Watanabe, R. P. M. Fortes, and A. L. Dias, "Using acceptance tests to validate accessibility requirements in RIA," in *Proceedings of the International Cross-Disciplinary Conference on Web Accessibility*, ser. W4A '12. Lyon, France: Association for Computing Machinery, Apr. 2012, pp. 1–10. [Online]. Available: <http://doi.org/10.1145/2207016.2207022>
- [15] —, "Acceptance tests for validating ARIA requirements in widgets," *Universal Access in the Information Society*, vol. 16, no. 1, pp. 3–27, Mar. 2017. [Online]. Available: <https://doi.org/10.1007/s10209-015-0437-9>
- [16] "Tenon.io." [Online]. Available: <https://tenon.io/>
- [17] "Access Assistant." [Online]. Available: <https://www.webaccessibility.com/>
- [18] "ARC Toolkit." [Online]. Available: <https://www.tpgi.com/arc-platform/arc-toolkit/>
- [19] "Accessibility Insights for Web - Microsoft." [Online]. Available: <https://accessibilityinsights.io/>
- [20] "WAVE Web Accessibility Evaluation Tool," <https://wave.webaim.org/>, 2020, accessed: 2020-08-25.
- [21] "W3C: Web Content Accessibility Guidelines (WCAG) 2.1," <https://www.w3.org/TR/WCAG21/>, updated: 2021-08-25.
- [22] "Google Material Design: Dialogs," <https://material.io/components/dialogs/web#using-dialogs>, accessed: 2021-07-04.
- [23] "Nielsen Norman Group: Modal Nonmodal Dialogs: When (When Not) to Use Them," <https://www.nngroup.com/articles/modal-nonmodal-dialog/>, updated: 2021-08-25.
- [24] Uaag 2.0 reference: Explanations, examples, and resources for user agent accessibility guidelines 2.0. [Online]. Available: <https://www.w3.org/TR/UAG20-Reference/>
- [25] "APG - Dialog (Modal)," <https://www.w3.org/WAI/ARIA/apg/patterns/dialogmodal/>, updated: 2022-09-01.
- [26] "W3C: Failure of Success Criterion 2.4.3 due to using dialogs or menus that are not adjacent to their trigger control in the sequential navigation order," <https://www.w3.org/WAI/WCAG21/Techniques/failures/F85>, updated: 2021-08-25.
- [27] "Techniques for WCAG 2.0: F85 - Failure of Success Criterion 2.4.3 due to using dialogs or menus that are not adjacent to their trigger control in the sequential navigation order," <https://www.w3.org/TR/2016/NOTE-WCAG20-TECHS-20161007/F85>, updated: 2020-11-14.
- [28] "WAI-ARIA Authoring Practices 1.1 : Dialog (Modal)," https://www.w3.org/TR/2018/NOTE-wai-aria-practices-1.1-20180726/#dialog_modal, accessed: 2022-09-01.
- [29] "WAI-ARIA Authoring Practices 1.1: Keyboard Interaction," <https://www.w3.org/TR/wai-aria-practices-1.1/#keyboard-interaction-7>, updated: 2020-11-14.
- [30] "U.S. GSA 18F Accessibility Guide: Keyboard access," <https://accessibility.18f.gov/keyboard/>, 2020, accessed: 2021-08-25.
- [31] "WAI-ARIA Authoring Practices 1.1 - Dialog Popup Keyboard Interaction," <https://www.w3.org/TR/wai-aria-practices/#dialog-popup-keyboard-interaction>, updated: 2021-08-25.
- [32] "Understanding WCAG 2.0: No Keyboard Trap - Understanding SC 2.1.2," <https://www.w3.org/TR/UNDERSTANDING-WCAG20/keyboard-operation-trapping.html>, updated: 2020-11-14.
- [33] "WebAIM: Keyboard Accessibility - Keyboard Testing," <https://webaim.org/techniques/keyboard/>, updated: 2022-09-01.
- [34] P. T. Chiou, A. S. Alotaibi, and W. G. J. Halfond, "Detecting and localizing keyboard accessibility failures in web applications," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 855–867. [Online]. Available: <https://doi.org/10.1145/3468264.3468581>
- [35] "Bootstrap: Bootstrap's JavaScript modal plugin," <https://getbootstrap.com/docs/4.0/components/modal/>, updated: 2020-11-09.
- [36] "Lightbox: The original lightbox script," <https://lokeshdhakar.com/projects/lightbox2/>, updated: 2020-11-09.
- [37] "MDN Web Docs - dialog: The Dialog element," <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/dialog>, accessed: 2021-08-24.
- [38] "Modal Dialog Design Pattern in WAI-ARIA Authoring Practices 1.1: Modal Dialog Example," <https://www.w3.org/TR/wai-aria-practices-1.1/examples/dialog-modal/dialog.html>, updated: 2019-08-14.
- [39] "CSS Positioned Layout Module Level 3: Detailed stacking context - Painting order," <https://www.w3.org/TR/2015/WD-css3-positioning-20150203/#painting-order>, updated: 2015-02-03.
- [40] "MDN Web Docs: The stacking context," https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Positioning/Understanding_z_index/The_stacking_context, updated: 2021-11-10.
- [41] "Nielsen Norman Group: Modal Nonmodal Dialogs: When (When Not) to Use Them," <https://www.nngroup.com/articles/modal-nonmodal-dialog/>, updated: 2017-04-23.
- [42] "DT: 100vw and the horizontal overflow you probably didn't know about," <https://destroytoday.com/blog/100vw-and-the-horizontal-overflow-you-probably-didnt-know-about>, updated: 2022-08-27.
- [43] "W3C: F85 - Failure of Success Criterion 2.4.3 due to using dialogs or menus that are not adjacent to their trigger control in the sequential navigation order," <https://www.w3.org/TR/WCAG20-TECHS/F85.html>, accessed: 2020-03-15.
- [44] LOTUS Project Web Site. <https://sites.google.com/usc.edu/lotus/home>.
- [45] "The Selenium Browser Automation Project: WebDriver - Keyboard," <https://www.selenium.dev/documentation/en/webdriver/keyboard/>, 2021, accessed: 2021-01-14.
- [46] "MDN Web Docs: Intersection Observer API," https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API, updated: 2021-11-20.
- [47] S. Roy Choudhary, M. R. Prasad, and A. Orso, "X-pert: Accurate identification of cross-browser issues in web applications," in *Proceedings of the 2013 International Conference on Software Engineering*, ser. ICSE '13. IEEE Press, 2013, p. 702–711.
- [48] "Moz's list of the most popular 500 websites on the internet," <https://moz.com/top500>, 2022, accessed: 2022-03-14.
- [49] M. Bajammal and A. Mesbah, "Semantic web accessibility testing via hierarchical visual analysis," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 2021, pp. 1610–1621.

- [50] "W3C: ARIA Authoring Practices Guide (APG) - Design pattern for modal dialogs," <https://www.w3.org/WAI/ARIA/apg/patterns/dialogmodal/>, 2022, accessed: 2022-08-31.
- [51] W3c: Techniques and failures for success criterion 2.4.3 - focus order. [Online]. Available: <https://www.w3.org/TR/UNDERSTANDING-WCAG20/navigation-mechanisms-focus-order.html#navigation-mechanisms-focus-order-techniques-head>
- [52] "Web Accessibility Evaluation Tools List," <https://www.w3.org/WAI/ER/tools/>.
- [53] N. Fernandes, D. Costa, S. Neves, C. Duarte, and L. Carriço, "Evaluating the accessibility of rich internet applications," in *Proceedings of the International Cross-Disciplinary Conference on Web Accessibility*, ser. W4A '12. Lyon, France: Association for Computing Machinery, Apr. 2012, pp. 1–4. [Online]. Available: <http://doi.org/10.1145/2207016.2207019>
- [54] "PowerMapper: SortSite - Accessibility Checker and Validator," <https://www.powermapper.com/products/sortsites/checks/accessibility-checks/>, updated: 2021-11-23.
- [55] "Accessible Rich Internet Applications (WAI-ARIA) 1.1: aria-modal (property)," <https://www.w3.org/TR/wai-aria-1.1/#aria-modal>, updated: 2021-11-24.
- [56] "Accessible Rich Internet Applications (WAI-ARIA) 1.1: aria-haspopup (property)," <https://www.w3.org/TR/wai-aria-1.1/#aria-haspopup>, updated: 2021-11-24.
- [57] S. Trewin, S. Yan, D. Margaretos, M. Gower, P. Jenkins, and C. Pandhi, "Feasibility of automatically assigning severity scores to web accessibility problems," in *Proceedings of the 20th International ACM SIGACCESS Conference on Computers and Accessibility*, ser. ASSETS '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 364–366. [Online]. Available: <https://doi.org/10.1145/3234695.3241001>
- [58] "Guidance on Web Accessibility and the ADA_doj." [Online]. Available: <https://beta.ada.gov/resources/web-guidance/>
- [59] E. Yigitbas, S. Sauer, and G. Engels, "Adapt-ui: An ide supporting model-driven development of self-adaptive uis," in *Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems*, ser. EICS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 99–104. [Online]. Available: <https://dl.acm.org/10.1145/3102113.3102144>
- [60] J. R. Escolar, "A context-aware dialog model for multi-device web development," in *Proceedings of the 5th ACM SIGCHI Symposium on Engineering Interactive Computing Systems*, ser. EICS '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 167–170. [Online]. Available: <https://dl.acm.org/10.1145/2494603.2480341>
- [61] Z. Sun, A. Stent, and I. V. Ramakrishnan, "Dialog generation for voice browsing," in *Proceedings of the 2006 International Cross-Disciplinary Workshop on Web Accessibility (W4A): Building the Mobile Web: Rediscovering Accessibility?*, ser. W4A '06. New York, NY, USA: Association for Computing Machinery, 2006, p. 49–56. [Online]. Available: <https://dl.acm.org/10.1145/1133219.1133228>
- [62] Y. Borodin, "Multi-layer dialog generation for non-visual web access," *SIGACCESS Access. Comput.*, no. 86, p. 20–23, sep 2006. [Online]. Available: <https://dl.acm.org/10.1145/1196148.1196153>
- [63] J. C. Hendy, J. Link, K. S. Booth, and J. McGrenere, "Parameter selection in keyboard-based dialog boxes," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ser. CHI '11. New York, NY, USA: Association for Computing Machinery, 2011, p. 2761–2764. [Online]. Available: <https://dl.acm.org/10.1145/1978942.1979350>
- [64] P. Panchekha, A. T. Geller, M. D. Ernst, Z. Tatlock, and S. Kamil, "Verifying that web pages have accessible layout," in *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2018. Philadelphia, PA, USA: Association for Computing Machinery, Jun. 2018, pp. 1–14. [Online]. Available: <http://doi.org/10.1145/3192366.3192407>
- [65] P. Panchekha, M. D. Ernst, Z. Tatlock, and S. Kamil, "Modular verification of web page layout," *Proceedings of the ACM on Programming Languages*, vol. 3, no. OOPSLA, pp. 151:1–151:26, Oct. 2019. [Online]. Available: <https://doi.org/10.1145/3360577>
- [66] A. Alshayban, I. Ahmed, and S. Malek, "Accessibility Issues in Android Apps: State of Affairs, Sentiments, and Ways Forward," 2020, p. 12.
- [67] A. S. Ross, X. Zhang, J. Fogarty, and J. O. Wobbrock, "An Epidemiology-inspired Large-scale Analysis of Android App Accessibility," *ACM Transactions on Accessible Computing*, vol. 13, no. 1, pp. 4:1–4:36, Apr. 2020. [Online]. Available: <http://doi.org/10.1145/3348797>
- [68] S. Chen, C. Chen, L. Fan, M. Fan, X. Zhan, and Y. Liu, "Accessible or Not An Empirical Investigation of Android App Accessibility," *IEEE Transactions on Software Engineering*, pp. 1–1, 2021, conference Name: IEEE Transactions on Software Engineering.
- [69] C. Vendome, D. Solano, S. Liñán, and M. Linares-Vásquez, "Can Everyone use my app? An Empirical Study on Accessibility in Android Apps," in *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Sep. 2019, pp. 41–52, iSSN: 2576-3148.
- [70] A. S. Alotaibi, P. T. Chiou, and W. G. Halfond, "Automated Repair of Size-Based Inaccessibility Issues in Mobile Applications," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Nov. 2021, pp. 730–742, iSSN: 2643-1572.
- [71] F. Mehralian, N. Salehnamadi, and S. Malek, "Data-driven accessibility repair revisited: on the effectiveness of generating labels for icons in Android apps," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, Aug. 2021, pp. 107–118. [Online]. Available: <https://doi.org/10.1145/3468264.3468604>
- [72] X. Zhang, A. S. Ross, and J. Fogarty, "Robust Annotation of Mobile Application Interfaces in Methods for Accessibility Repair and Enhancement," in *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology*, ser. UIST '18. New York, NY, USA: Association for Computing Machinery, Oct. 2018, pp. 609–621. [Online]. Available: <http://doi.org/10.1145/3242587.3242616>
- [73] A. S. Alotaibi, P. T. Chiou, and W. G. Halfond, "Automated Detection of TalkBack Interactive Accessibility Failures in Android Applications," in *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*, Apr. 2022, pp. 232–243, iSSN: 2159-4848.
- [74] J. Wu, X. Zhang, J. Nichols, and J. P. Bigham, "Screen Parsing: Towards Reverse Engineering of UI Models from Screenshots," in *The 34th Annual ACM Symposium on User Interface Software and Technology*, ser. UIST '21. New York, NY, USA: Association for Computing Machinery, Oct. 2021, pp. 470–483. [Online]. Available: <https://doi.org/10.1145/3472749.3474763>